VYAPARAI • SYSTEMS, DEMO EXECUTION & DEFENSE TALKING POINTS

CANDIDATE: VENKATESH PAI | CO-LEAD SYSTEMS ENGINEER

Team Shadow (ID: HW26-F08) | 1st Year B.Tech AIML, KIT Kolhapur | Problem Statement #3: Local Vendor Inventory Digitization

Live Production: https://vyaparai-oddf.onrender.com/ | GitHub: https://github.com/awateatharva4-pixel/vyaparai

MODULE 1: SYSTEMS ENGINEERING LAYER & INGRESS DECOUPLING

The Systems Challenge: WhatsApp and Meta webhooks enforce a strict **5-second HTTP timeout**. If backend AI inference takes 3-4 seconds, any network jitter causes socket termination and Meta fires aggressive retry storms (sending duplicate requests). If unhandled, this causes catastrophic duplicate sales!
Sub-50ms Asynchronous Ingress: Venkatesh engineered ingress decoupling using FastAPI BackgroundTasks. The webhook receives the payload, authenticates HMAC SHA-256 signatures, hands off work to an async background worker, and returns **HTTP 200 OK in < 35ms**.
Thread-Safe Idempotency Ledger: Every incoming message ID is inserted into an IdempotencyRecord table with a **UNIQUE(message_id)** constraint. Duplicate deliveries trigger a database integrity violation, rolling back instantly with zero double-deductions.

MODULE 2: DUAL-ROLE FIREWALL, PACKING QUEUE & WEBSOCKET STREAMING

1. Dual-Role Security Firewall & Zero Privilege Escalation:
• *The Threat:* Both shopkeeper and neighborhood customers message the exact same WhatsApp bot number. What stops a mischievous customer from typing *'undo last sale'* or *'5 kg sugar cancel kara'* to tamper with the merchant's financial books?
• *Firewall Invariant:* In app/core/config.py, AUTHORIZED_MERCHANT_PHONES = {'919579869506', '917558455021'} strictly fences admin rights. is_merchant_sender() normalizes WhatsApp prefixes (+, -, spaces, @c.us).
• *Zero Privilege Escalation:* Customer numbers sending admin commands are safely sandboxed: they receive a polite customer greeting and the merchant ledger is untouched.

2. Remote 'Yaadi' Live Packing Queue & Zero Direct Deduction:
• *The Invariant:* Inbound customer grocery chits MUST NOT decrement stock upon receipt. If they did, fake lists could deplete the entire store catalog!
• *The Staging Queue:* Inbound yaadi orders are saved in **PENDING** status in Supabase Cloud PostgreSQL / SQLite WAL.
• *Live Merchant Dashboard:* Emits **NEW_CUSTOMER_YAADI** over WebSockets in < 200ms. Dashboard mounts a live order card with audio chime.
• *One-Click Pack & Deduct:* Merchant reviews the paper chit, packs items, and clicks 'Pack & Deduct'. Only then does the backend commit **Transaction(type='SALE')** and decrement stock. Double-packing attempts return HTTP 400 Bad Request (Idempotency).

3. Reactive Real-Time WebSockets & Durable Reboot Snapshot:
• *Real-Time WebSocket Hub:* In app/core/websocket.py, ConnectionManager manages active UI clients and broadcasts events: CATALOG_ONBOARDED, NEW_CUSTOMER_YAADI, CUSTOMER_ORDER_PACKED, METRICS_UPDATE.
• *Durable State Snapshotting:* On cloud hosting (Render), free containers sleep. InventoryService.save_state_snapshot atomically flushes all products, customer credits, and orders to data/vyaparai_state.json. On reboot, seed_initial_catalog(force=False) rehydrates the entire state with sequence sync.

VYAPARAI • VENKATESH'S STAGE DEMO CUE SHEET

SECOND-BY-SECOND LIVE STAGE EXECUTION & CONTINGENCY TIMELINE

MODULE 3: VENKATESH'S LIVE STAGE DEMO CUE SHEET (SECOND-BY-SECOND)

Time / Cue	Action on Phone / Dashboard	Backend Execution Trigger	What the Judges See on Screen
T + 00s Hook	Atharva addresses judges; Venkatesh places phone on table.	Dashboard open at /dashboard on projector screen.	Live inventory ledger showing 10 default Kolhapur Kirana SKUs.
T + 15s Video Sweep	Venkatesh records 5-second video sweep across judges' desk (bottle, pen, book, biscuit) and sends to WhatsApp bot.	Green-API videoMessage webhook receives MP4; queues async Gemini video decomposition.	Instant WhatsApp ack: *'Video prapt jhala! Dukan scan karat aahe...'*
T + 25s Zero-Touch	Gemini extracts 4 items; commits to DB; emits CATALOG_ONBOARDED over WebSocket.	Backend generates collision-proof SKUs (AUTO-VID-XXXX) and bilingual voice aliases.	**BAM!** Audio chime rings. 4 glowing product cards pulse into the dashboard. Store valuation increases live!
T + 45s Voice Sale	Atharva speaks WhatsApp voice note: *'2 pen dile ani 20 chi sakhar'* (Rs. 20 sugar + 2 pens).	Reverse-math calculates 476g sugar in 0.02ms; pen stock drops from 2.0 to 0.0.	WhatsApp scale card arrives: **Kaatyavar theva: 476 grams**. Pen card flashes red: Low-stock alert!
T + 65s Yaadi Chit	Venkatesh (from customer phone) sends photo of handwritten paper chit with grocery list.	Dual-role firewall detects customer; Gemini Vision extracts items; saves as **PENDING** order.	Dashboard chimes! **New Customer Yaadi** appears in queue with itemized checklist and 'Pack & Deduct' button.
T + 80s Pack & Climax	Atharva clicks **'Pack & Deduct'** on dashboard.	Backend transitions order to **PACKED**; commits ACID sale; decrements stock atomically.	Order card turns emerald green; WhatsApp receipt sent to customer; merchant ledger updated!

MODULE 4: VENKATESH'S CODE NAVIGATION CHEAT SHEET (WHERE TO POINT)

Target Component	Exact File & Function	Key Systems Invariant	What Venkatesh Demonstrates
Dual-Role Firewall	`core/config.py` `is_merchant_sender`	Role separation by phone; Zero privilege escalation	Fences +91 9579869506 and +91 75584 55021 to merchant ledger; sandboxes customer senders.
Webhook Ingress Gateway	`routers/webhook.py` `handle_whatsapp_webhook`	Async decoupling < 35ms; Green-API / Meta / Twilio	Dispatches videoMessage, imageMessage, and text to BackgroundTasks; returns 200 OK instantly.
Customer Yaadi Router	`routers/webhook.py` `background_process_customer_yaadi`	Chit OCR, ungrounded item filter, PENDING staging	Processes customer photo, calculates total excluding unstocked items, broadcasts NEW_CUSTOMER_YAADI.
Packing Queue API	`routers/simulator.py` `pack_order_endpoint`	Idempotent packing commit; Zero direct deduction	Transitions PENDING to PACKED; decrements stock only on merchant click; rejects double-pack with 400.
Real-Time Dashboard UI	`templates/dashboard.html` `ws.onmessage handlers`	Reactive DOM diffing, glowing cards, audio chime	Mounts new video items live; animates packing queue; triggers Web Audio API synthesis.

VYAPARAI • VENKATESH'S DEFENSE DOSSIER: TOP 6 JUDGE TRAPS

SYSTEMS, CONCURRENCY & SECURITY REBUTTALS AGAINST CSE EVALUATORS

MODULE 5: TOP 6 BRUTAL JUDGE TRAPS & SPOKEN REBUTTALS

Q1: What happens if two sales occur at the exact same millisecond? What prevents race conditions?
'Sir, standard SQLite deferred transactions allow concurrent readers to observe stale stock before acquiring write locks, leading to lost updates. We solved this at the engine level in app/core/database.py: we configure SQLite in Write-Ahead Logging (WAL) mode with busy_timeout=5000 and hook into SQLAlchemy before_cursor_execute to inject BEGIN IMMEDIATE on all write transactions. This immediately reserves an exclusive database lock, serializing concurrent sales. Furthermore, our Supabase Cloud PostgreSQL pool uses row-level locking (SELECT FOR UPDATE) to guarantee ACID serializability.'

Q2: What if the college Wi-Fi blocks WhatsApp webhooks or external ports during your demo?
'Sir, we anticipated network instability in the CSE lab. In app/routers/simulator.py, we engineered an offline judge simulator bench at /simulator. It features an authentic mobile chat mockup, role switcher (Merchant vs Customer), 1-click vernacular test chips, drag-and-drop video sweep ingestion, and paper chit photo upload. The entire pipeline executes locally with zero external network dependency, allowing judges to test every feature without their phones.'

Q3: What prevents a malicious customer from draining a shop's stock with fake high-volume orders?
'Sir, this is our Zero Direct Deduction invariant. When a customer sends a grocery list or chit, our system creates the order in PENDING status. Inventory is NOT decremented upon receipt. The order is placed into the merchant dashboard packing queue. Only when the physical shopkeeper verifies the items and clicks Pack & Deduct does the system commit the sale and decrement stock. Fake customer orders never lock or deplete physical store stock.'

Q4: Can a customer see the merchant's profit margins, purchase costs, or admin commands?
'Sir, absolute zero privilege escalation is enforced. In app/core/config.py, only authenticated merchant phone numbers can query financial metrics, margins, or trigger stock restocks. When a customer phone sends an admin command like 'margins dakhva' or 'undo', our router intercepts it at the boundary, sandboxes it, and returns a standard customer greeting. Financial data and margin fields are completely stripped from customer payloads.'

Q5: How does your video onboarding handle serverless or container restarts where local files are wiped?
'Sir, on cloud platforms like Render, ephemeral storage is wiped when containers restart. We solved this with our Durable State Snapshot Engine in InventoryService.save_state_snapshot. Every time a catalog is onboarded, all products, customer credit accounts, and transaction records are atomically flushed to data/vyaparai_state.json. On boot, seed_initial_catalog automatically rehydrates the full database state and resynchronizes PostgreSQL sequences.'

Q6: What prevents duplicate transactions if WhatsApp retries a webhook delivery?
'Sir, we engineered a thread-safe Idempotency Ledger. Our database maintains an IdempotencyRecord table with a UNIQUE(message_id) index. When a webhook arrives, our background worker executes a pre-flight insert of the incoming message ID. If WhatsApp retries delivery due to network latency, the database raises an IntegrityError, which our code intercepts and silently discards, guaranteeing zero duplicate sales deductions.'